

데이터 구조 진단 설계서

1. 개요

1.1 목적

수집된 데이터 모델 스냅샷과 실제 운영 DBMS의 현재 스키마를 비교하여 변경사항(GAP)을 파악하는 기능. 구조 진단은 "수집 시점과 현재 DB가 얼마나 다른가"만 책임지며, 표준 준수 여부 판단은 기존 표준 진단의 역할로 남긴다.

1.2 기존 기능과의 관계

기능	역할	범위
데이터 모델 수집 (기존)	DBMS 카탈로그 → TB_DATA_MODEL_ATTR 저장	수집만
구조 진단 (신규)	수집 스냅샷 vs 실제 DBMS → diff	변경 감지만
표준화 진단 (기존)	수집된 컬럼 vs TB_TERMS 비교	표준 준수 검증

각 기능은 독립적으로 실행 가능하며, 선후관계를 강제하지 않는다.

1.3 메뉴 구조

데이터 구조 진단	← 독립 대메뉴
└ 구조 진단	(1차 구현 완료)
└ 스키마 비교 뷰어	(2차 개선 대상)

2. 1차 구현 (완료)

2.1 기능

- 데이터모델 선택 → "구조 진단 실행"
- q-executor가 백그라운드에서 실제 DBMS에 접속, 현재 스키마를 읽고 가장 최신 수집 스냅샷과 diff
- 결과: ADDED / MODIFIED / DELETED 컬럼 목록
- 스탯 카드, 변경유형 필터, 진단 이력

2.2 제약 (2차에서 해결)

- 수집 이력이 여러 건 있어도 **항상 최신 수집과만 비교**
- diff 결과만 보여주고, 수집 데이터와 실제 DB를 **나란히 비교하는 뷰가 없음**

3. 2차 개선 사항

3.1 수집 이력 선택 기능

현재: 데이터모델 선택 → 자동으로 최신 수집건과 비교 **변경:** 데이터모델 선택 → **수집 이력 선택** → 해당 시점의 스냅샷과 현재 DB 비교

표준화 진단에서 수집 일시를 선택하는 것과 동일한 패턴.

[데이터모델 ▼] [수집일시 ▼] [구조 진단 실행]

수집일시 드롭다운 예시:

20260331 15:00:19 (최신)
20260330 10:30:22
20260325 09:15:00

효과: - 과거 특정 시점의 수집 데이터와 현재 DB를 비교할 수 있음 - "3월 25일에 수집한 것 기준으로 지금 DB가 얼마나 바뀌었나?" 확인 가능 - 기본값은 최신 수집건 (현재 동작과 동일)

백엔드 변경: - `/api/std/structDiag/execute` 에 `clctId` 파라미터 추가 (선택) - `clctId` 가 있으면 해당 수집건 사용, 없으면 최신 수집건 사용 (하위호환)

3.2 스키마 비교 뷰어 (신규 화면)

구조 진단이 **변경된 것만** 보여준다면, 스키마 비교 뷰어는 **수집 데이터와 실제 DB를 나란히 전체** 보여주는 화면.

화면 레이아웃

스키마 비교 뷰어

[데이터모델 ▼] [수집일시 ▼] [비교 실행] [변경만 보기 ☐

테이블 목록

TB_USER	● 동일
TB_ORDER	● 동일
TB_LOG	◐ 변경
TB_NEW_TBL	○ 신규
TB_OLD_TBL	✕ 삭제

42개 테이블

컬럼 비교

컬럼명	수집 스냅샷	현재 DB
USER_ID	V(20)	V(20)
USER_NM	V(50)	V(100)
EMAIL	(없음)	V(200)
OLD_FLAG	C(1)	(없음)

15개 컬럼

범례: ● 동일 ◐ 변경 ○ 신규(DB에만) ✕ 삭제(수집에만)

좌측: 테이블 목록 패널

- 전체 테이블 목록 (수집 + 현재 DB 합집합)
- 각 테이블 옆에 상태 아이콘:
- ● **동일**: 수집과 현재 DB의 컬럼이 완전 일치
- ◐ **변경**: 컬럼 추가/삭제/변경이 있음
- ○ **신규**: 현재 DB에만 존재 (수집 시점에 없었음)
- ✕ **삭제**: 수집에만 존재 (현재 DB에서 삭제됨)
- 테이블 클릭 → 우측에 해당 테이블의 컬럼 비교 표시
- 상태별 필터 (전체 / 변경만 / 신규만 / 삭제만)

우측: 컬럼 비교 그리드

- 선택한 테이블의 컬럼을 **수집 스냅샷 vs 현재 DB** 나란히 표시
- 컬럼 구성:

컬럼명	수집 타입	수집 길이	수집 Nullable	현재 타입	현재 길이	현재 Nullable	상태
USER_ID	VARCHAR	20	N	VARCHAR	20	N	동일
USER_NM	VARCHAR	50	Y	VARCHAR	100	Y	길이 변경
EMAIL	-	-	-	VARCHAR	200	Y	신규
OLD_FLAG	CHAR	1	Y	-	-	-	삭제

- 행 색상: 동일=기본, 변경=주황 배경, 신규=초록 배경, 삭제=빨강 배경
- "변경만 보기" 체크박스로 동일한 컬럼 숨기기 가능

데이터 흐름

비교 실행 클릭

- 1. 선택한 수집건의 스냅샷 조회 (TB_DATA_MODEL_ATTR)
- 2. 실제 DBMS 접속 → 현재 스키마 읽기 (기존 구조 진단과 동일)
- 3. 테이블 단위로 그룹핑, 컬럼별 매칭
- 4. 전체 결과를 프론트에 반환 (변경 여부 관계없이 전부)

구조 진단과의 차이

항목	구조 진단	스키마 비교 뷰어
목적	변경사항 빠르게 파악	전체를 나란히 상세 비교
결과	변경된 것만	전체 (동일 포함)
표시	플랫 테이블 (변경 목록)	좌우 분할 (테이블 선택 → 컬럼 비교)
용도	"뭐가 바뀌었나?" 빠른 확인	"정확히 어디가 어떻게 다른가?" 상세 확인

4. API 설계

4.1 기존 API (1차)

Method	URL	설명
POST	/api/std/structDiag/execute	구조 진단 실행 (executor 위임)
GET	/api/std/structDiag/status/{diagId}	진단 상태 폴링
GET	/api/std/structDiag/history	진단 이력 목록
GET	/api/std/structDiag/result/{diagId}	진단 결과 상세
GET	/api/std/structDiag/latestStatus	벋지용 최신 상태

4.2 2차 추가 API

Method	URL	설명
POST	/api/std/structDiag/execute	clctId 파라미터 추가 (선택적)
POST	/api/std/structDiag/compareSchema	스키마 비교 뷰어용: 전체 테이블+컬럼 반환

compareSchema 응답 구조

```

{
  "tables": [
    {
      "tableNm": "TB_USER",
      "status": "MODIFIED",
      "columns": [
        {
          "columnNm": "USER_ID",
          "status": "SAME",
          "snapshot": { "dataType": "VARCHAR", "dataLen": 20, "nullable": "N" },
          "current": { "dataType": "VARCHAR", "dataLen": 20, "nullable": "N" }
        },
        {
          "columnNm": "USER_NM",
          "status": "MODIFIED",
          "snapshot": { "dataType": "VARCHAR", "dataLen": 50, "nullable": "Y" },
          "current": { "dataType": "VARCHAR", "dataLen": 100, "nullable": "Y" }
        },
        {
          "columnNm": "EMAIL",
          "status": "ADDED",
          "snapshot": null,
          "current": { "dataType": "VARCHAR", "dataLen": 200, "nullable": "Y" }
        }
      ]
    }
  ],
  "summary": {
    "totalTables": 42,
    "sameTables": 38,
    "modifiedTables": 2,
    "addedTables": 1,
    "deletedTables": 1
  }
}

```

5. 기존 코드 활용 계획

구현 항목	기존 자산	추가 개발
DBMS 접속 + 스키마 읽기	StructDiagService의 DBMS 읽기 로직 그대로	없음
수집 이력 선택	DSDataDiag.vue의 수집일시 드롭다운 패턴 복사	프론트 UI 추가
비교 뷰어 테이블 목록	없음	신규 Vue 컴포넌트 (좌우 분할)
비교 뷰어 API	StructDiagService diff 로직 변형 (전체 반환)	compareSchema API
diff 로직	StructDiagService.toAttrMap + diff	SAME 상태 추가

6. 표준 진단과의 연계 (느슨한 연계)

구조 진단과 표준 진단은 독립적이지만, 표준 진단 화면에서 구조 진단 상태를 뱃지로 표시하여 데이터 신뢰도를 안내한다.

6.1 표준 진단 화면의 데이터소스/스키마 선택 영역

quality 스키마	✓ 구조 검증됨 (2026-03-31 15:50)	← 초록 뱃지
hr 스키마	⚠ 구조 검증 3일 경과 (2026-03-27)	← 주황 뱃지
legacy 스키마	(구조 미검증)	← 뱃지 없음

6.2 뱃지 판정 기준

상태	조건	표시
검증됨 (초록)	가장 최근 구조 진단에서 변경사항 0건 + 7일 이내	✓ 구조 검증됨 (일시)
경과 (주황)	구조 진단 이력은 있지만 7일 초과	⚠ 구조 검증 N일 경과
변경 감지 (빨강)	가장 최근 구조 진단에서 변경사항 있음	⊖ 구조 변경 감지 (N건)
미검증 (회색)	구조 진단 이력 없음	(표시 없음)

7. 구현 우선순위

1차 (완료)

순서	항목	상태
1	DDL (TB_STRUCT_DIAG_HISTORY, DETAIL)	✓
2	StructDiagService (executor, DBMS 라이브 접속 + diff)	✓
3	StructDiagController (center/executor)	✓
4	DSStructDiag.vue (진단 실행 + 결과 + 이력)	✓
5	대메뉴 분리 (데이터 구조 진단)	✓

2차 (예정)

순서	항목	필수 여부
1	수집 이력 선택 드롭다운 (프론트 + 백엔드 clctId 파라미터)	필수
2	스키마 비교 뷰어 백엔드 (compareSchema API)	필수
3	스키마 비교 뷰어 프론트 (좌우 분할 화면)	필수
4	스키마 비교 뷰어 메뉴 등록	필수
5	표준 진단 화면에 구조 검증 बै지 표시	권장